

# Contents

|                                                              |          |
|--------------------------------------------------------------|----------|
| <b>Week 3 — Foundations: Tools, Context &amp; Evaluation</b> | <b>1</b> |
| Learning objectives                                          | 1        |
| 1. Tool use and function calling                             | 1        |
| 2. The Model Context Protocol (MCP)                          | 3        |
| 3. Context management                                        | 6        |
| 4. Research and benchmarking methodology for agents          | 7        |
| 5. Discussion questions for this session                     | 8        |
| Proposal pitches (this session)                              | 9        |
| Further reading                                              | 9        |

## Week 3 — Foundations: Tools, Context & Evaluation

**CO5151 · Advanced Agentic Artificial Intelligence** Learning outcomes touched: L.O.1.2, L.O.2.1 · Feeds directly into: S1-2 (Tool Use & MCP), S1-8 (Evaluation & Benchmarking), and Project requirement R2/R5 · Duration: ~149 minutes (includes proposal pitches in the last ~45 minutes.md §3). This > content's session number shifted — it is taught in the real **Week 2** (Wed 16 Sep 2026), not Week 3 > — because Week 1 and Week 2 of the original 15-session design are now combined into one real Week 1. > The content itself is unchanged.

### Learning objectives

By the end of this session, students can:

1. Explain how native function calling works mechanically (schema → constrained generation → execution → observation) and identify the failure modes specific to *irreversible* or *expensive* tools.
2. Describe the Model Context Protocol's architecture (host/client/server, the four primitives, the initialize handshake) and explain what problem it solves that per-tool custom integrations do not.
3. Name at least three concrete context-management techniques and the failure mode each one addresses.
4. Design a credible evaluation for an agent system: choose a metric, justify a baseline, and specify what must be controlled for (seeds, cost, contamination) to make the result trustworthy.
5. Defend a project proposal against exactly this methodology bar in the pitch segment of this session.

---

## 1. Tool use and function calling

### 1.1 From free text to constrained action

An LLM's native output is unstructured text. For an agent to *act* — call an API, run a query, execute code — that text has to become a valid, executable call: a specific function name and a specific set of typed arguments. Three generations of approach to this problem, worth distinguishing because they trade off differently on reliability, and because your project will use the third but should understand why the earlier two existed:

- **Toolformer** (Schick et al., NeurIPS 2023) — a model *fine-tuned* to insert special tool-call tokens into its own generated text at the positions where a tool would help (e.g., a calculator call inline in an arithmetic sentence), self-supervised by checking whether inserting the call improves the model's own prediction of the following tokens. Elegant, but tool use is baked into one model's weights at training time — not something you can add a new tool to without retraining.
- **ToolLLM** (Qin et al., ICLR 2024) — scales tool use to a very large number of real-world APIs (16000+) via instruction-tuning on synthesized multi-tool-use trajectories, plus a depth-first search decision tree for

tool *selection* when the number of candidate tools is too large to consider by brute force in one prompt. This is the first point in the timeline where “which tool, out of thousands, should I even consider” becomes its own sub-problem, not just “how do I call the tool correctly.”

- **Native function calling** (current commercial LLM APIs) — the model provider trains the base model itself to reliably emit a structured call (typically JSON matching a provided schema) whenever given a list of available tool schemas in its context, with no extra fine-tuning by the developer. This is what nearly every framework you’ll use for your project calls under the hood.

## 1.2 Native function calling, mechanically

You provide the model with a **schema per tool** at inference time — name, description, and a typed parameter specification:

```
{
  "name": "search_flights",
  "description": "Search for available flights between two cities on a given date.",
  "parameters": {
    "type": "object",
    "properties": {
      "origin": {"type": "string", "description": "IATA airport code, e.g. 'SGN'"},
      "destination": {"type": "string", "description": "IATA airport code, e.g. 'HAN'"},
      "date": {"type": "string", "description": "ISO 8601 date, e.g. '2026-09-14'"}
    },
    "required": ["origin", "destination", "date"]
  }
}
```

At generation time, the model either produces ordinary text or, when it decides a tool is needed, produces a structured call matching one of the provided schemas (search\_flights with concrete argument values). The calling application is responsible for validating the arguments, actually executing the call against the real API, and appending the result back into context as an *Observation* — this is exactly the `execute(decision.action, tools)` step in the Week 1 §2.2 pseudocode, now made concrete.

**What the schema’s description field is actually doing:** it is the *entire* interface the model has to decide when and how to use the tool — there is no other channel. A vague or ambiguous description causes wrong tool selection or malformed arguments just as surely as a bug in your own code would; writing tool descriptions is a design task, not documentation, and — as previewed here and covered in depth in S1-2 and S2-3 — it is also the largest **security surface** in the whole agent: any text that ends up inside a tool description or a tool’s *result* is exactly as “trusted” by the model as your own system prompt, which is the root mechanism behind indirect prompt injection.

## 1.3 Error handling, idempotency, and irreversibility

Three properties of a tool call that must be designed for explicitly, because an LLM will not reliably infer them on its own from a plain function signature:

- **Idempotency** — does calling the tool twice with the same arguments have the same effect as calling it once? A `get_weather` call is idempotent; a `charge_card` call is not. Agents retry on failure (network errors, malformed arguments, rate limits) by design — a non-idempotent tool called twice by accident during a retry is a real, common failure mode, not a hypothetical one.
- **Reversibility** — can the *effect* of the call be undone? Sending an email cannot be undone; adding an item to a draft order usually can be. Week 2’s critique of ReAct’s cost assumption (§2.2 of that note) applies

here directly: a system designed assuming cheap, reversible actions will treat an irreversible action with the same casualness, which is a design defect, not a quirk of one paper.

- **Error surfacing** — when a tool call fails, what does the agent see? A raw stack trace, a structured error the model can reason about and retry differently, or (worse) a silent failure that looks like a valid empty result? Structured, actionable error messages are as much a part of “tool design” as the happy-path schema.

For your own project (Requirement R2), each tool’s permission level should explicitly record whether it is read-only, reversible-write, or irreversible-write — this is not extra paperwork, it is the first input to the threat model you’ll build for Requirement R6.

## 1.4 The Berkeley Function-Calling Leaderboard, briefly

Worth knowing as a pointer, not memorizing: an evaluation suite (extension reading for S1-2, alongside Patil et al.’s **Gorilla**) that specifically measures function-calling *correctness* — does the model choose the right function, with the right arguments, in the right format — separately from whether the overall task succeeds. This separation matters: a system can fail a task either because the model chose the wrong action, or because it chose the right action but formatted the call incorrectly, and these call for different fixes (better tool descriptions vs. better prompting/parsing/retries respectively).

---

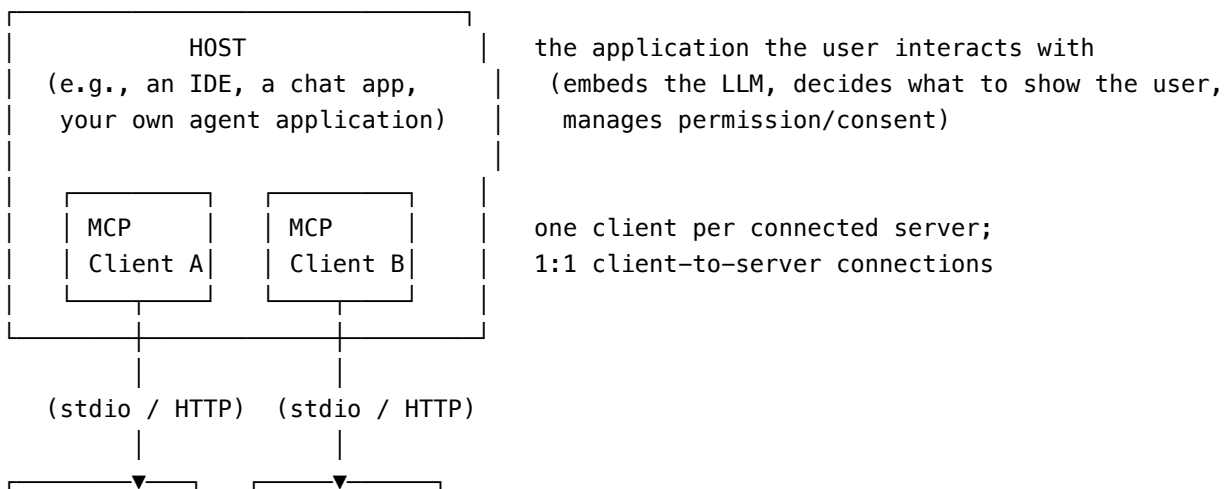
## 2. The Model Context Protocol (MCP)

### 2.1 The problem MCP solves

Before a standard protocol, every agent framework that wanted to connect an LLM to, say, a database, a filesystem, and a ticketing system had to write its own custom integration code for each pairing — an  $M \times N$  problem:  $M$  agent frameworks, each needing custom code for  $N$  tools/data sources. Anthropic’s Model Context Protocol (announced 2024) standardizes the *interface* between an LLM application and external tools/data, turning  $M \times N$  custom integrations into  $M + N$ : each tool/data source implements one MCP server once, and each agent application implements one MCP client once, and any client can talk to any server.

This is the same category of move as USB standardizing peripheral connectors, or HTTP standardizing how browsers talk to servers — MCP does not make agents smarter; it makes tool integration a solved, reusable engineering problem rather than bespoke glue code per pairing.

### 2.2 Architecture: host, client, server



|                                    |                                               |
|------------------------------------|-----------------------------------------------|
| MCP Server<br>(e.g. a<br>database) | MCP Server<br>(e.g. a<br>ticketing<br>system) |
|------------------------------------|-----------------------------------------------|

each server exposes tools/resources/prompts for ONE system – a database, a filesystem, an internal API, etc.

- The **host** is the user-facing application; it owns the LLM conversation and — critically for security — is where user consent and permission decisions should be enforced (never trust the model alone to decide whether an action needs approval).
- Each **client** manages exactly one stateful connection to one server.
- Each **server** exposes a specific system’s capabilities through four primitives (below), and knows nothing about which LLM or which other servers are involved.

## 2.3 The four primitives

| Primitive        | Who controls it                                                            | Analogy                                                                     | Example                                                                                          |
|------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Tools</b>     | Model-controlled (the LLM decides when to call, host may require approval) | A function the model can call                                               | <code>query_database(sql)</code> ,<br><code>create_ticket(title, body)</code>                    |
| <b>Resources</b> | Application-controlled (host decides what to attach to context)            | A file the model can read                                                   | <code>file:///project/README.md</code> ,<br><code>db://customers/row/482</code>                  |
| <b>Prompts</b>   | User-controlled (user explicitly selects a template)                       | A reusable slash-command/template                                           | <code>/summarize-ticket</code> ,<br><code>/draft-reply</code>                                    |
| <b>Sampling</b>  | Server-initiated                                                           | The server asking the <i>host’s</i> LLM to generate something on its behalf | A server needing a natural-language summary generated using the host’s model rather than its own |

The three-way split between *model-controlled*, *application-controlled*, and *user-controlled* primitives is itself a security design decision, not an implementation detail — it is the protocol’s built-in acknowledgment that not every decision about what enters context should be delegated to the model. Keep this distinction in mind for S2-3/S2-4: many injection defenses are really proposals to move more decisions from “model-controlled” to “application-controlled.”

## 2.4 The handshake and a tool call, concretely

A simplified message flow (MCP uses JSON-RPC 2.0 under the hood):

```
Client → Server: initialize { protocolVersion, clientCapabilities }
Server → Client: initialize (result) { protocolVersion, serverCapabilities, serverInfo }
Client → Server: notifications/initialized

Client → Server: tools/list
Server → Client: { tools: [ { name: "query_database", description: "...", inputSchema: {...} }, ... ] }
```

... the host's LLM decides to call `query_database(sql="SELECT ...")` ...

Client → Server: `tools/call { name: "query_database", arguments: {sql: "SELECT ..."} }`

Server → Client: `{ content: [ {type: "text", text: "<query result>" } ], isError: false }`

Notice that `tools/list` is exactly the schema-advertising step from §1.2, now standardized and served dynamically by the tool provider itself rather than hand-written into your agent's prompt — the host's LLM sees whatever tools the server currently advertises, which can change at runtime.

## 2.5 Transports and capability negotiation

MCP is transport-agnostic: a server can be a local subprocess communicating over **stdio** (simple, no network exposure, good for local tools) or a remote service reached over **HTTP** (with server-sent events for streaming), each with different trust implications — a local stdio server inherits the host process's own privileges directly, while a remote HTTP server is a genuine network boundary you must reason about separately. The `initialize` handshake's capability negotiation lets a client and server agree on which optional protocol features (e.g., streaming, resource subscriptions) both sides support before any real work begins — standard protocol-design practice, worth recognizing here so it isn't mysterious when you see it in an actual server implementation.

## 2.6 Security considerations, previewed

This is a preview of S2-3/S2-4, deliberately placed here because a security-blind mental model of MCP will produce a security-blind project architecture. Three things to internalize now:

- **A tool's description field is untrusted input from the model's perspective.** If a server (or anything that can influence what a server returns, e.g., a poisoned database row) can control what text appears in a tool description or a tool result, it can influence the model's behavior exactly as if that text were part of the user's own prompt. This is the mechanism, not a hypothetical.
- **Scoping and consent belong in the host, not the model.** The protocol supports fine-grained, per-tool consent; whether your agent actually asks the user before an irreversible tool call is an application design choice MCP enables but does not enforce for you.
- **A “confused deputy” risk:** an agent with legitimate access to multiple servers (e.g., email + a public web-search tool) can be tricked, via content encountered through one server, into misusing its legitimate access to another — this compound risk is exactly what Willison's “lethal trifecta” framing (assigned in S2-3) names: (1) exposure to untrusted content, (2) access to private data, and (3) an ability to externally communicate, present *together* in one agent.

## 2.7 MCP vs. A2A, briefly

MCP standardizes **agent-to-tool/data** communication. Google's **Agent2Agent (A2A)** protocol (2025, extension reading in S1-2 and S2-2) instead standardizes **agent-to-agent** communication — how one autonomous agent discovers, delegates to, and exchanges structured messages with another, independently operated agent, which is a different problem (peer coordination) than “how do I call this one tool.” The two are complementary, not competing: a multi-agent system (S2-1) plausibly uses A2A between agents and MCP from each agent to its own tools.

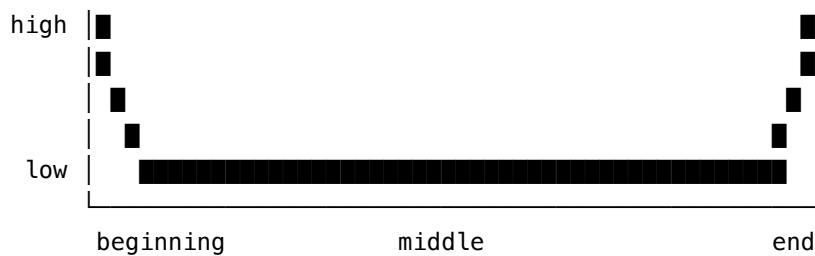
### 3. Context management

#### 3.1 The window is finite, and position matters

Everything an agent “knows” at decision time must fit in the model’s context window — the system prompt, conversation history, tool schemas, retrieved documents, and prior tool observations all compete for the same finite budget. Two distinct problems follow from this, and it’s worth keeping them separate:

- **Capacity:** the window has a hard token limit; long-running agents will eventually exceed it no matter how the content is organized.
- **Positional degradation** (“lost in the middle,” Liu et al., TACL 2024): even *within* the window’s capacity, models are empirically less accurate at using information placed in the *middle* of a long context than information near the beginning or end — so simply “fitting” content is not sufficient; *where* critical information sits matters independently of whether it technically fits.

Accuracy at retrieving a fact, by its position in a long context (illustrative shape, not exact numbers):



This shape is the direct, practical reason “just put everything relevant into a giant context window” is not a substitute for the memory and retrieval design covered in S1-3/S1-4 — a fact being *present* in context is not the same as the model reliably *using* it.

#### 3.2 Techniques, and the failure mode each addresses

| Technique                            | What it does                                                                                                                                                                    | Addresses                                                                                                                                                              |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compaction / summarization</b>    | Periodically replace older conversation turns with a generated summary                                                                                                          | Capacity — keeps the running context under the window limit over a long-lived session                                                                                  |
| <b>Scratchpads</b>                   | A designated, explicitly-labeled area of context for the model’s own intermediate working (distinct from the “final answer” area)                                               | Reasoning quality — same underlying idea as CoT (Week 2 §1), applied as a context-management discipline rather than a one-off prompt trick                             |
| <b>Retrieval instead of stuffing</b> | Store material outside the context window (a vector or keyword index) and pull in only the pieces relevant to the current step, on demand                                       | Both capacity and positional degradation — retrieved content can be placed favorably (e.g., near the end, right before the question) rather than buried mid-context    |
| <b>Structured context sections</b>   | Explicitly label/delimit distinct kinds of content (system instructions vs. retrieved documents vs. tool results vs. user input) rather than one undifferentiated block of text | Confusability — reduces the chance the model treats retrieved/tool content as an instruction (a partial mitigation for the injection risk in §2.6, not a complete one) |

None of these is a substitute for the others — a production agent typically compacts old history, retrieves relevant material on demand rather than stuffing everything permanently into context, and still uses a scratchpad-style reasoning discipline for the current step. S1-3 (Agent Memory) covers the *persistent* half of this problem (what survives across sessions and how it’s written/retrieved); this section is specifically about managing the *current* context window’s finite, position-sensitive budget.

---

## 4. Research and benchmarking methodology for agents

### 4.1 Why this goes before you build, not after

This section exists in Week 3 — before Seminar 1 begins and before most groups have working code — deliberately: an evaluation plan designed *after* a system is built tends to be shaped to flatter whatever the system already does well. Designing the evaluation plan alongside the architecture (as Project Requirement R5 and the D1 proposal both require) is a discipline, not a formality.

### 4.2 Metrics: task success, pass@k, and cost/latency together

- **Task success rate:** the fraction of task instances the agent completes correctly, under a defined correctness check (exact match, a unit test, a rubric-scored judgment). Meaningless without stating *how* correctness is checked — an LLM-judged “success” and a unit-test-verified “success” are different metrics wearing the same name.
- **pass@k:** given k independent attempts per task instance, the fraction of instances where *at least one* of the k attempts succeeds. Useful for measuring raw capability with retries allowed, but it silently assumes you have a reliable way to *pick* the successful attempt in a real deployment — which is often exactly the unsolved part of the problem (see the Reflexion evaluator discussion in Week 2 §3.2).
- **pass<sup>k</sup>** (Yao et al.,  $\tau$ -bench, 2024): given k independent attempts, the fraction where *all* k attempts succeed — a much stricter reliability measure, directly motivated by the observation that a tool-using agent deployed in production is called repeatedly, and a technique that succeeds 90% of the time on any *one* attempt can still be unreliable in the sense that matters for deployment (it fails at least once within a handful of calls with high probability). Prefer pass<sup>k</sup> over pass@k whenever you are making a claim about production reliability rather than raw one-shot capability.
- **Cost and latency, always reported alongside success:** a technique that improves success rate by spending 10× the tokens (recall self-consistency and Tree-of-Thoughts from Week 2 §4.3) is not unconditionally “better” — it is a different point on a cost-quality curve, and a paper (or your own project report) that reports accuracy without cost is not making a complete comparison.

### 4.3 Seeds, variance, and ablations

- **Seeds/variance:** LLM outputs are stochastic (even at temperature 0, some providers’ sampling is not perfectly deterministic across calls). A single run’s success rate is a point estimate with unknown variance; reporting results from  $\geq 3$  independent runs (as Project Requirement R5 requires) and showing the spread is the minimum needed to distinguish a real effect from noise.
- **Ablations:** to support a claim like “the memory module improves performance,” you need a comparison against the *same system with the memory module removed*, holding everything else fixed — not a comparison against an entirely different baseline system that also differs in other ways. An ablation isolates one variable; a baseline comparison against an unrelated system does not.

#### 4.4 Contamination and the LLM-as-judge problem

- **Contamination:** many public benchmarks (or data very similar to them) may already be present in a model’s pretraining data, inflating reported performance in a way that has nothing to do with the technique being tested. Prefer recently-released or held-out task sets where feasible, and be explicit in your report about what you could and could not control for.
- **LLM-as-judge:** using an LLM to score another LLM’s output (common when human grading doesn’t scale) is convenient but introduces its own, separate biases — documented preferences for longer answers, for answers similar in style to the judge’s own outputs, and sensitivity to the order options are presented in. If you use an LLM judge (for your project or in critiquing a paper that does), state which model, what prompt, and — ideally — validate the judge against a small human-labeled sample rather than trusting it blindly.

#### 4.5 Reproducibility packaging

A result that cannot be reproduced by someone else running your code is not yet a result the field can build on — this is precisely Project Requirement R8. Concretely: pin dependency versions, record seeds, log the exact model version/date used (models change over time even at a fixed name), and provide a one-command path from a clean checkout to the reported numbers. “It worked when I ran it” is not the same claim as “it is reproducible,” and a report should not conflate the two.

#### 4.6 What a credible project evaluation looks like (Requirement R5, made concrete)

Putting §4.2–4.5 together, the minimum bar for your project’s evaluation:

- A task set of at least 20 instances, from a defined source (an existing benchmark subset, or your own curated set with a stated construction method).
- At least one stated success metric with an explicit correctness check, plus cost and latency reported alongside it.
- At least two system configurations compared (e.g., with vs. without a component, or your system vs. one baseline) — this is your ablation.
- At least 3 independent runs/seeds, with variance shown, not just a single run’s number.
- An error analysis: for the failures, a categorization of *why* they failed (wrong tool call, context overflow, hallucinated fact, evaluator too strict/lenient) — this is usually the most informative part of an evaluation section and the part most often skipped.

---

### 5. Discussion questions for this session

1. For a tool your project will expose (§1.3): is it read-only, reversible-write, or irreversible-write? What changes in your architecture if you got that classification wrong?
2. MCP’s three-way split of tools/resources/prompts by *who controls them* (§2.3) — pick one primitive and describe a scenario where using the wrong one (e.g., making something model-controlled that should be user-controlled) creates a security or usability problem.
3. “Lost in the middle” (§3.1) suggests position matters independently of capacity. If you had to choose between (a) a shorter context with critical facts near the end, or (b) a longer context containing strictly more information but with critical facts buried in the middle, which would you expect to perform better, and what would make you wrong?
4. Pick one paper you’ve read so far this semester and identify which of  $\text{pass}@1$  /  $\text{pass}@k$  /  $\text{pass}^k$  it reports (or should have reported, but didn’t) — does the choice match the claim the paper is making?



## Proposal pitches (this session)

The final ~45 minutes of this session are project proposal pitches (5' per group).<sup>2</sup>/Appendix B.1 for the proposal template and grading. Bring §4.6's checklist with you: a proposal whose evaluation plan cannot yet satisfy those five bullet points at least in outline is exactly the kind of “changes required” feedback to expect.

## Further reading

- Schick et al. — *Toolformer: Language Models Can Teach Themselves to Use Tools* (NeurIPS 2023).
- Qin et al. — *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs* (ICLR 2024).
- Patil et al. — *Gorilla: Large Language Model Connected with Massive APIs* (NeurIPS 2024).
- Anthropic — *Introducing the Model Context Protocol* (2024) and the MCP specification (modelcontextprotocol.io) — read the specification's own security/authorization sections directly.
- Google — *Announcing the Agent2Agent Protocol (A2A)* (2025).
- Liu et al. — *Lost in the Middle: How Language Models Use Long Contexts* (TACL 2024).
- Yao et al. —  *$\tau$ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains* (2024) — the pass<sup>k</sup> metric in §4.2.
- Kapoor et al. — *AI Agents That Matter* (arXiv 2024 / TMLR 2025) — methodology critique underlying most of §4.